OS - Orange Problem-2

NAME: J KARTHIK

SRN: PES1UG25AM804

COURSE: Operating Systems

SECTION: B

ABSTRACT

This project implements a simplified version control system (PES-VCS) inspired by Git. It supports file tracking, staging, commit creation, and history traversal using content-addressable storage.

AIM

To design and implement a simplified version control system supporting file tracking, staging, commit creation, and history traversal.

THEORY

A Version Control System tracks changes in files over time. Content-addressable storage ensures integrity using hashing.

```
karthik@karthik-VirtualBox:~$ git clone https://github.com/jkarthik10/PES1UG25AM
804-pes-vcs.git
Cloning into 'PES1UG25AM804-pes-vcs'...
remote: Enumerating objects: 23, done.
remote: Counting objects: 100% (23/23), done.
remote: Compressing objects: 100% (23/23), done.
remote: Total 23 (delta 5), reused 6 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (23/23), 30.04 KiB | 134.00 KiB/s, done.
Resolving deltas: 100% (5/5), done.
karthik@karthik-VirtualBox:~$ cd PES1UG25AM804-pes-vcs
karthik@karthik-VirtualBox:~/PES1UG25AM804-pes-vcs$
```

```
karthik@karthik-VirtualBox:~/PES1UG25AM804-pes-vcs$ make clean
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_obj
ects.o test_tree.o
rm -rf .pes
```

```
karthik@karthik-VirtualBox:~/PES1UG25AM804-pes-vcs$ git push origin main
Username for 'https://github.com': jkarthik10
Password for 'https://jkarthik10@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 6 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 310 bytes | 310.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/jkarthik10/PES1UG25AM804-pes-vcs.git
   46e02b5..e6b4f9f  main -> main
```

IMPLEMENTATION

Phase 1: Object Storage

Implemented functions:
- object_write()
- object_read()

Features:
- SHA-256 hashing
- Directory sharding
- Integrity verification

Screenshot 1A:

```
karthik@karthik-VirtualBox:~/PES1UG25AM804-pes-vcs$ gcc test_objects.c object.o -o test_objects -lcrypto
karthik@karthik-VirtualBox:~/PES1UG25AM804-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B:

```
karthik@karthik-VirtualBox:~/PES1UG25AM804-pes-vcs$ chmod +x test_sequence.sh
karthik@karthik-VirtualBox:~/PES1UG25AM804-pes-vcs$ ./test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: f38ddf7806eb... Initial commit

Log after first commit:
commit f38ddf7806eb5f019b17e46752c00a00a3b926bc10ffea1046728cf80bee355b
Author: J Karthik <PES1UG25AM804>
Date:   1776766045

    Initial commit
```

```
Author: J Karthik <PES1UG25AM804>
Date:   1776766045

    Update file.txt

commit f38ddf7806eb5f019b17e46752c00a00a3b926bc10ffea1046728cf80bee355b
Author: J Karthik <PES1UG25AM804>
Date:   1776766045

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
89f0a6b3425e940e8633ef06e89ca9bd31346f461d1919b14f0dd59b8f8302e0

--- Object Store ---
Objects created:
7
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/5f/49c1216583bbf5da06769bb8c19c974c8d813346be11a441f22f259c142efb
.pes/objects/89/f0a6b3425e940e8633ef06e89ca9bd31346f461d1919b14f0dd59b8f8302e0
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/f3/8ddf7806eb5f019b17e46752c00a00a3b926bc10ffea1046728cf80bee355b

=== All integration tests completed ===
```

Phase 2: Tree Objects

Implemented:

- tree_from_index()

Features:

- Converts index into hierarchical tree
- Supports directory structure

Screenshot 2A:

Screenshot 2B:

```
vboxuser@os2:~/os/PES1UG25AM816-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
vboxuser@os2:~/os/PES1UG25AM816-pes-vcs$ xxd .pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
00000000: 626c 6f62 2031 3800 4475 706c 6963 6174  blob 18.Duplicat
00000010: 6520 636f 6e74 656e 740a                 e content.
```

Phase 3: Index (Staging Area)

Implemented:
- index_load()
- index_save()
- index_add()

Features:

- Tracks staged files
- Stores metadata

Screenshot 3A:

```
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  pes.c
  untracked:  Makefile
  untracked:  test_tree.c
  untracked:  index.h
  untracked:  test_sequence.sh
  untracked:  test_objects.c
  untracked:  commit.h
  untracked:  .gitignore
  untracked:  README.md
  untracked:  commit.c
  untracked:  tree.h
  untracked:  pes.h
  untracked:  test_tree
  untracked:  tree.c
  untracked:  test_objects
  untracked:  index.c
  untracked:  object.c
```

Screenshot 3B:

```
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776367658 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776367722 6 file2.txt
```

Phase 4: Commit System

Implemented:
- commit_create()

Features:
- Creates commit objects
- Maintains history
- Updates HEAD

Screenshot 4A:

```
vboxuser@os2:~/os/PES1UG25AM816-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776367658 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776367722 6 file2.txt
```

Screenshot 4B:

```
vboxuser@os2:~/os/PES1UG25AM816-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/75/840ac20f8e24184dc026fafb9139d90b06eeedf445219288088f1cc5c4f8e0
.pes/objects/c3/314a3c90233937183a44d6cd029c94a44644ee3a3065b4c4edbe76c041b1a4
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/refs/heads/main
```

Screenshot 4C:

```
vboxuser@os2:~/os/PES1UG25AM816-pes-vcs$ cat .pes/refs/heads/main
c3314a3c90233937183a44d6cd029c94a44644ee3a3065b4c4edbe76c041b1a4
vboxuser@os2:~/os/PES1UG25AM816-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Final Output:

```
vboxuser@os2:~/os/PES1UG25AM816-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: b07713f3d1db... Initial commit

Log after first commit:
commit b07713f3d1db1d213196091e0b3c23246f6220451edcebd7b72860d56ee5c005
Author: PES User <pes@localhost>
Date:   1776369499

    Initial commit
```

```
--- Second Commit ---
Committed: b0696b99dd99... Update file.txt

--- Third Commit ---
Committed: 425a0a6e77bd... Add farewell

--- Full History ---
commit 425a0a6e77bda094e05d3d270079e574652db71709b7696364b92fef6bb4d0b6
Author: PES User <pes@localhost>
Date:   1776369499

    Add farewell

commit b0696b99dd99c46280a01ad747ceec541ef821fd223f11a5926cb7f5fab55a5d
Author: PES User <pes@localhost>
Date:   1776369499

    Update file.txt

commit b07713f3d1db1d213196091e0b3c23246f6220451edcebd7b72860d56ee5c005
Author: PES User <pes@localhost>
Date:   1776369499

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
425a0a6e77bda094e05d3d270079e574652db71709b7696364b92fef6bb4d0b6

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/23/e6e45234c8b612bd97d02f11db0b74ed10dfea726e7515c78c80e324044a93
.pes/objects/42/5a0a6e77bda094e05d3d270079e574652db71709b7696364b92fef6bb4d0b6
.pes/objects/42/5a0a6e77bda094e05d3d270079e574652db71709b7696364b92fef6bb4d0b6
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/8b/4f581e473f22720d10a0d19f1b1935c43aa49d983c26e61b71a84137d12582
.pes/objects/b0/696b99dd99c46280a01ad747ceec541ef821fd223f11a5926cb7f5fab55a5d
.pes/objects/b0/7713f3d1db1d213196091e0b3c23246f6220451edcebd7b72860d56ee5c005
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/bf/5bebbb151f3c9896d57284a43efee7032adb94098b01b1a24399295f7f8829
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

ANALYSIS QUESTIONS

Q5.1: Implementation of pes checkout <branch>

A branch in the PES-VCS system is represented as a file in .pes/refs/heads/ that stores the hash of the latest commit. Implementing 'pes checkout <branch>' involves multiple steps:
1. Branch Validation: Check whether the branch file exists.
2. Read Target Commit: Extract commit hash from the branch file.
3. Update HEAD: Modify .pes/HEAD to point to the selected branch.
4. Reconstruct Working Directory: Read commit → tree → blobs and recreate files.
5. Update Index: Replace index contents with the new tree structure.

Q5.2: Detecting a Dirty Working Directory

A dirty working directory means there are uncommitted changes. Detection involves:
1. Compare working files with index using size and timestamp.
2. Compare index with committed tree using hashes.
3. Detect conflicts if both differ.
4. Abort operation to prevent data loss.

Q5.3: Detached HEAD State

A detached HEAD occurs when HEAD directly points to a commit hash instead of a branch. In this state, commits can still be made but are not associated with any branch. These commits may be lost if not referenced.

Recovery methods:
1. Create a new branch from the commit.
2. Manually update references.
3. Use commit hash directly.

GARBAGE COLLECTION

Q6.1: Algorithm for Removing Unreachable Objects

1. Identify root references from all branches.
2. Traverse commit graph (DFS/BFS).
3. Traverse trees and collect blobs.
4. Mark all reachable objects.
5. Delete objects not in reachable set.

Q6.2: Garbage Collection Race Condition

A race condition may occur when garbage collection runs alongside commit creation. This can lead to deletion of newly created objects before they are referenced.

Prevention:
1. Use locking mechanisms.
2. Two-phase garbage collection.
3. Delay deletion (grace period).
4. Track active references.